TRABALHO: SIMULADOR DE ESCALONAMENTO MULTIPROCESSADO COM PAGINAÇÃO SOB DEMANDA

Disciplina: Sistemas Operacionais
Conteúdo: Escalonamento de CPU, memória principal e memória virtual.
Equipe: trios | **Entrega: 03/11/26 até as 19:00** |

1. Contexto

Escalonamento de CPU e gerenciamento de memória costumam ser estudados em capítulos separados, mas no sistema real eles interagem o tempo todo: um processo que sofre *page fault* sai da CPU, e a escolha de quem executa muda quais páginas estão na memória.

Desenvovler um simulador de um sistema operacional simplificado, que recebe uma carga de 1000 processos e permite observar, por meio de métricas, como diferentes políticas de escalonamento e de memória se comportam em máquinas com 1 e com 4 CPUs.

2. Objetivos

2.1 Objetivo geral

Desenvolver um simulador de escalonamento de CPU integrado a paginação sob demanda e, com base em dados coletados por vocês, analisar criticamente o comportamento dos algoritmos em diferentes cenários de carga e de hardware.

2.2 Objetivos específicos

1. Implementar os escalonadores FCFS, SJF (não preemptivo) e Round Robin (RR) com quantum variável.
2. Simular máquinas com 1 CPU e com 4 CPUs compartilhando uma única fila de prontos.
3. Simular memória física paginada com page fault e substituição FIFO e LRU.
4. Modelar o custo da troca de contexto e entender seu efeito no desempenho.
5. Acompanhar e registrar os conjuntos de processos (prontos, executando, bloqueados, terminados) ao longo da simulação.
6. Coletar, calcular e interpretar métricas de desempenho, justiça e uso de recursos.
7. Produzir o diagrama de Gantt da janela de tempo de 0 a 200.

8. Investigar, com os dados, fenômenos como injustiça, starvation, efeito comboio e thrashing, caso ocorram.
9. Justificar conclusões com dados e teoria, e reconhecer as limitações do modelo.

3. Conceitos que vocês devem dominar antes de programar

Conceito	Descrição
Processo	Programa em execução. Aqui, cada processo alterna trechos de CPU e de E/S
Burst de CPU	Trecho contínuo em que o processo precisa da CPU
E/S	Trecho em que o processo espera um dispositivo (disco, rede). Não usa CPU
Fila de prontos	Processos que têm tudo o que precisam e só aguardam uma CPU
Preempção	Tirar um processo da CPU antes de ele terminar o burst
Troca de contexto	Salvar o estado do processo que sai e carregar o do que entra. Aqui é um custo fixo de tempo
Página e quadro	Página é um bloco do processo. Quadro é um bloco da memória física que comporta uma página
Page fault	A página acessada não está em nenhum quadro e precisa ser trazida do disco
Localidade de referência	Tendência de um programa acessar repetidamente as mesmas páginas em curto intervalo
Carga do sistema	Trabalho que chega por unidade de tempo ÷ trabalho que a máquina processa por unidade de tempo. Ex.: chegam em média 3,8 unidades de CPU por unidade de tempo numa máquina de 4 CPUs (capacidade 4) → carga de 95%

4. Requisitos gerais

Requisito	Descrição
Linguagem	Livre (Python, Java, C++ ou outra), desde que o professor consiga executar
Escalonadores	FCFS, SJF (não preemptivo) e RR, todos no mesmo programa
Configurações	--cpus 1 e --cpus 4, com a mesma carga
Seleção na execução	Escalonador, quantum, CPUs, política de memória e quadros escolhidos por argumentos (ou menu), sem editar ou recompilar código
Entrada	Arquivos JSON
Saída	Resumo padronizado (seção 9), Gantt (seção 9.2) e log opcional (seção 9.3)

Requisito	Descrição
Repositório	Código postado, instruções de execução e resultados reproduzíveis
Desempenho	O simulador será executado n de vezes. Escolham estruturas de dados que mantenham a execução em poucos minutos.

5. A carga de trabalho

Cada grupo recebe dois arquivos, gerados a partir de uma seed própria.

Arquivo	Descrição
cenario_A_t0.json	1000 processos, **todos chegando em t = 0**
cenario_B_espacadas.json	1000 processos com **chegadas espalhadas no tempo**, com carga média próxima de 95% da capacidade de 4 CPUs

O mesmo arquivo é usado em 1 e em 4 CPUs. O arquivo descreve apenas a **carga**. A máquina (CPUs, quadros, escalonador) é definida pelos argumentos. Rodar a mesma carga nas duas configurações é o que torna válida a comparação e o cálculo do speedup.

5.1 Características gerais

- Os processos são curtos (maioria) ou longos (minoria), numa distribuição bimodal.
- Todos são mistos: alternam CPU e E/S.
- Cada processo tem um número de páginas e uma sequência de referências a elas.

5.2 Formato do arquivo (JSON)

JSON porque as listas aninhadas (bursts, referências) são lidas diretamente por bibliotecas padrão. Python tem json nativo. Em Java, use Gson ou Jackson. Em C++, use a biblioteca de arquivo único nlohmann/json.

CONFIGURAÇÃO DOS DADOS


```
{
  "pid": 17,
  "chegada": 342,
  "classe": "curto",
  "paginas": 6,
  "bursts": [
    {"cpu": 4, "refs": [0, 1, 1, 2]},
    {"es": 12},
    {"cpu": 3, "refs": [2, 3, 3]}
  ]
}
```


Campos do processo

Campo	**Significado**
pid	Identificador do processo
chegada	Instante em que o processo entra no sistema
classe	curto ou longo. É apenas um **rótulo para análise**, e o escalonador **não deve usá-lo** para decidir
paginas	Quantidade de páginas do processo (numeradas de 0 a paginas−1)
bursts	Lista, em ordem, das etapas da vida do processo. Começa e termina com burst de CPU

Etapas dentro de bursts

Etapa	Significado
{"cpu": N, "refs": [...]}	O processo precisa de **N unidades de tempo de CPU**. A lista refs tem N valores: na i-ésima unidade de CPU executada, o processo acessa a página refs[i]
{"es": M}	O processo fica **M unidades de tempo** esperando um dispositivo de E/S. Não usa CPU nem faz referências

5.3 Atenção: dois “4” diferentes

	Significado	Onde está
"cpu": 4	O processo precisa de 4 **unidades de tempo** de processamento	No arquivo (carga)
--cpus 4	A máquina simulada tem 4 **núcleos**	Na linha de comando (máquina)

Um processo é sequencial: **usa um núcleo por vez** e nunca é dividido entre núcleos. Os núcleos extras só permitem que **processos diferentes** executem ao mesmo tempo.

5.4 Exemplo: vida do processo 17 (1 CPU, sem concorrência e sem page faults)

Tempo	O que acontece
342	Chega e entra na fila de prontos
342 a 346	Executa 4 unidades, acessando as páginas 0, 1, 1, 2
346 a 358	E/S de 12 unidades (não usa CPU, não conta como espera)
358 a 361	Executa 3 unidades, acessando as páginas 2, 3, 3
361	Termina

Neste caso simplificado: CPU total = 7, E/S = 12, espera = 0, turnaround = 361 − 342 = 19.

5.5 Exemplo: o mesmo processo 17 com page faults

Suposições: 1 CPU, memória inicialmente vazia, nenhum outro processo, troca de contexto ignorada, page fault = 20 unidades.

Tempo	Referência	O que acontece
342	página 0	**Fault.** Bloqueado de 342 a 362
362 a 363	página 0	Acerto. 1ª unidade de CPU
363	página 1	**Fault.** Bloqueado de 363 a 383
383 a 384	página 1	Acerto. 2ª unidade
384 a 385	página 1	Acerto (já estava na memória). 3ª unidade
385	página 2	**Fault.** Bloqueado de 385 a 405
405 a 406	página 2	Acerto. 4ª unidade. Fim do burst
406 a 418		E/S de 12 unidades
418 a 419	página 2	Acerto (nada a retirou da memória)
419	página 3	**Fault.** Bloqueado de 419 a 439
439 a 441	página 3	Acertos. 2ª e 3ª unidades. Termina em 441

Turnaround = 441 − 342 = 99 = 7 (CPU) + 12 (E/S) + 80 (4 faults × 20).

Importante: o arquivo não informa onde ocorrem page faults. Eles são consequência do estado da memória durante a simulação. Os processos possuem páginas, mas nenhuma está na memória ao chegar.

6. Regras do modelo

Todos os grupos devem seguir exatamente estas regras.

6.1 Tempo e estados

- O tempo é discreto, em unidades inteiras, começando em 0.
- Cada processo está sempre em um destes conjuntos:

Estado	Descrição
Novo	Ainda não chegou (chegada maior que o tempo atual)
Pronto	Na fila de prontos, aguardando uma CPU
Executando	Em uma CPU (inclui o instante de troca de contexto que o precede)
Bloqueado	Em E/S ou aguardando a resolução de um page fault
Terminado	Concluiu o último burst

- **Bloqueados não formam fila.** Cada um espera o seu próprio tempo, em paralelo. Não há fila de disco.
- Só a fila de prontos tem disputa, e é nela que se mede a **espera**.

6.2 Fila de prontos e despacho

- Existe uma única fila de prontos global, compartilhada por todas as CPUs.
- Quando uma CPU fica livre, ela pega um processo da fila segundo o escalonador. Se várias CPUs estiverem livres ao mesmo tempo, a de menor índice é atendida primeiro.
- O processo sai da fila de prontos no instante em que é despachado (inclusive durante a troca de contexto).
- **Ordem de eventos no mesmo instante t:**
    1. entram na fila as chegadas e os retornos de E/S e de page fault, em ordem de pid;
    2. o processo que sofreu preempção por quantum vai para o fim da fila;
    3. ocorrem os despachos.

6.3 Escalonadores

Escalonador	Regra
FCFS	Atende na ordem de entrada na fila de prontos.
SJF	Quando uma CPU fica livre, escolhe o processo pronto cujo **próximo burst de CPU (ou a parte que falta dele)** é o menor. Empate: o que entrou primeiro na fila. Uma vez despachado, o processo **não é retirado** da CPU por causa da chegada de outro menor
RR	Fila circular com quantum Q (--quantum). Ao fim do quantum, se há outro processo pronto, o atual vai para o fim da fila. Se **não** há outro pronto, continua sem custo e recebe novo quantum

Regras comuns:

- Ao voltar de E/S ou de page fault, o processo entra na fila de prontos como se fosse "novo" (no FCFS e no RR, no fim da fila).
- Se o burst termina antes do fim do quantum, a CPU é liberada imediatamente.
- Um page fault durante o quantum interrompe a execução e o restante do quantum é perdido.
- O SJF usa o valor do campo cpu, nunca o campo classe. No sistema real esse valor seria desconhecido e precisaria ser estimado. Registrem isso em "Limitações".

6.4 Custo da troca de contexto (Convenção A)

Ao despachar um processo para uma CPU, se ele for diferente do último processo que essa CPU executou, a CPU gasta 1 unidade de tempo em troca de contexto antes de começar a executá-lo. Isso vale também para o primeiro despacho de cada CPU. Se for o mesmo processo, não há custo.

- Durante a troca, a CPU está **ocupada**, mas nenhum processo avança.
- O custo é parametrizável (--custo-troca, padrão 1).
- Cada CPU deve acumular três parcelas: **tempo útil**, **tempo de troca** e **tempo ocioso**. A soma das três é igual ao makespan.

Situação	Há custo?
Fim do quantum no RR, fila vazia, o mesmo processo continua	Não

Situação	Há custo?
Fim do quantum no RR, existe outro processo pronto	Sim
Processo termina e há outro pronto	Sim
Processo termina e não há ninguém pronto	Não (a CPU fica ociosa)
Processo volta de E/S e a CPU não executou ninguém nesse intervalo	Não (é o mesmo processo)

6.5 Memória e page fault

Regra	Definição
Memória física	N quadros globais compartilhados pelas CPUs (--quadros, padrão 256)
O que é um quadro	Uma posição que guarda a identidade de uma página. O simulador não modela bytes, endereços nem offsets
Identificação de página	Par (pid, número da página). A página 0 de P1 e a página 0 de P2 são diferentes
Estrutura de dados	Livre, desde que respeite estas regras. O relatório deve descrevê-la e justificá-la
Início	Memória vazia. A página existe no disco desde o início, e o fault representa o tempo de trazê-la para um quadro
Referências	1 por unidade de CPU executada, na ordem de refs
Acerto	A página está em um quadro. A execução segue
Page fault	A página não está na memória. O processo é bloqueado por 20 unidades e a CPU fica livre. A unidade de CPU daquela referência não é consumida na detecção
Retorno do fault	O processo entra na fila de prontos. Ao executar de novo, repete a mesma referência, que agora é acerto. Até lá, a página fica protegida de substituição

Regra	Definição
Quadro livre	Se houver, a página é carregada nele
Memória cheia	Substituição **global**: a vítima é escolhida entre as páginas não protegidas de **qualquer** processo
Sem vítima	Se todos os quadros estiverem protegidos, o processo permanece bloqueado e **tenta de novo após 20 unidades**. Esse fault é contado uma vez
FIFO	Vítima = a página carregada há mais tempo
LRU	Vítima = a página com a referência mais antiga
Durante a E/S	As páginas do processo **permanecem** na memória e podem ser vítimas
Término	Os quadros do processo são liberados
Troca de contexto	Não altera a memória

Parâmetro: --subst fifo ou --subst lru.

7. Exemplos para validar o simulador

Rodem estes exemplos **no papel e no simulador** antes de usar os arquivos de 1000 processos.

7.1 Sem memória (1 CPU, FCFS, custo de troca = 1)

Processo	Chegada	Burst de CPU
P1	0	5
P2	0	3
P3	2	2

Intervalo	Evento
0 a 1	Troca de contexto para P1
1 a 6	P1 executa
6 a 7	Troca para P2
7 a 10	P2 executa
10 a 11	Troca para P3
11 a 13	P3 executa

Processo	Espera	Resposta	Turnaround
P1	0	0	6
P2	6	6	10
P3	8	8	11

CPU 1: útil = 10, troca = 3, ocioso = 0. Makespan = 13.Como exercício, refaçam com SJF e com RR (quantum 2).

7.2 Com page faults e fila (1 CPU, FCFS, troca = 0, fault = 5)

Processo	Chegada	Burst de CPU	Referências
P1	0	2	[0, 1]
P2	1	2	[0, 1]
P3	2	1	[0]

Memória grande o bastante para que nenhuma página seja retirada.

Tempo	Evento
0	P1 despachado. (P1,0): **fault**, bloqueado até 5. CPU sem pronto: ociosa
1	P2 chega, despachado. (P2,0): **fault**, bloqueado até 6
2	P3 chega, despachado. (P3,0): **fault**, bloqueado até 7
5 a 6	P1 volta, é despachado e executa a 1ª unidade (acerto)
6	P2 volta e entra na fila. P1 referencia (P1,1): **fault**, bloqueado até 11. P2 é despachado e executa de 6 a 7
7	P3 volta. P2 referencia (P2,1): **fault**, bloqueado até 12. P3 despachado, executa de 7 a 8 e **termina em 8**
8 a 11	CPU ociosa (todos bloqueados)
11 a 12	P1 volta e executa. **Termina em 12**
12 a 13	P2 volta e executa. **Termina em 13**

Processo	Espera	Turnaround
P1	0	12
P2	0	12
P3	0	6

Com todos bloqueados em faults, a fila quase não se forma e a CPU fica ociosa.

7.3 Para a fila aparecer (1 CPU, troca = 0, fault = 20)

Processo	Chegada	Burst de CPU	Referências
P1	0	4	[0, 0, 0, 0]
P2	0	3	[0, 0, 0]
P3	0	2	[0, 0]

Os três dão fault em t=0 e ficam bloqueados **em paralelo**. Em t=20 voltam juntos e a fila se forma.

Escalonador	Ordem	Espera P1 / P2 / P3	Espera média
FCFS	P1 (20–24), P2 (24–27), P3 (27–29)	0 / 4 / 7	3,67
SJF	P3 (20–22), P2 (22–25), P1 (25–29)	5 / 2 / 0	2,33

Os 20 unidades do fault **não entram** na espera, pois o processo estava bloqueado e não na fila de prontos.

8. Requisitos de execução

Argumento	Valores
--arquivo	Caminho do JSON
--cpus	1 ou 4
--sched	fcfs, sjf, rr
--quantum	Inteiro (usado só no RR)
--subst	fifo ou lru
--quadros	Inteiro (padrão 256)
--custo-troca	Inteiro (padrão 1)
--log	(opcional) gera o log da seção 9.3

Na defesa o professor pedirá mudanças desses parâmetros. Se for preciso alterar o código para isso, o requisito **não foi cumprido**.

9. Saídas do simulador

9.1 Resumo padronizado (ao final de cada execução)

1. Parâmetros usados.

2. Métricas globais obrigatórias (seção 10.1).
3. Tabela por CPU: tempo útil, tempo de troca, tempo ocioso e respectivas porcentagens.
4. Page faults totais e taxa de faults.

9.2 Gantt

- Cada execução gera um arquivo com o Gantt da janela **0 a 200**, com **uma faixa por CPU** (1 faixa com 1 CPU, 4 faixas com 4 CPUs).
- Cada intervalo indica o pid em execução, **TROCA** ou **OCIOSA**.

cpu,inicio,fim,evento

0,0,1,TROCA

0,1,6,P1

...

- No relatório, o Gantt deve aparecer como **figura legível**, uma por configuração.

9.3 Log de estados (para depuração e defesa)

Com --log, o simulador deve registrar, a cada instante em que algo muda (chegada, despacho, fim de burst, fault, retorno de E/S, término):

- o **evento** ocorrido e o pid envolvido;
- a **fila de prontos** (em ordem);
- o **conjunto de bloqueados**, com o instante de retorno de cada um;
- o processo em cada CPU;
- a quantidade de quadros ocupados.

Esse log é a ferramenta para provar, na defesa, que o simulador segue as regras.

10. Métricas

Os termos abaixo têm definição fixa para todos os grupos. Todas as obrigatórias usam só soma, média, máximo e divisão.

10.1 Obrigatórias

Métrica	O que é	Como calcular
Tempo de espera (média e máximo)	Tempo que o processo ficou na fila de prontos, aguardando CPU. Não inclui E/S nem page fault	Somar todos os intervalos na fila de prontos. Depois tirar a média e o maior valor entre os 1000 processos
Turnaround (média)	Tempo total que o processo passou no sistema, do começo ao fim	Instante de término − instante de chegada
Tempo de resposta (média)	Quanto o processo demora para começar a ser atendido pela primeira vez	Instante do primeiro despacho − instante de chegada
Makespan	Duração total da carga	Instante em que o último processo termina
Speedup	Quantas vezes 4 CPUs foram mais rápidas que 1. O ideal seria 4	Makespan com 1 CPU ÷ makespan com 4 CPUs (mesmo escalonador, quantum e memória)
Eficiência	Quanto do ganho ideal foi aproveitado. 1,0 é o perfeito	Speedup ÷ 4
Trocas de contexto	Quantas vezes o custo de troca foi cobrado	Contar as ocorrências (total e por CPU)
Utilização e tempo ocioso por CPU	Utilização é a fração do tempo em que a CPU esteve ocupada. Ocioso é o tempo sem processo para executar	Utilização = (tempo útil + tempo de troca) ÷ makespan. Ocioso = makespan − útil − troca
Page faults (total)	Quantos page faults ocorreram na execução	Contar as ocorrências

Exemplo de cálculo (seção 7.1): P1, P2 e P3 têm espera 0, 6 e 8. Espera média = (0 + 6 + 8) ÷ 3 = 4,67, e espera máxima = 8. Turnaround 6, 10 e 11 → média = 9. Makespan = 13. Utilização da CPU = (10 + 3) ÷ 13 = 100%.

11. Experimentos mínimos

Item	Mínimo exigido
Cenários	A e B
Configurações	1 CPU e 4 CPUs
Escalonadores	FCFS, SJF e RR
Quantums no RR	2,10 e 50
Memória	Execuções base com **LRU** e 256 quadros
FIFO × LRU	Em pelo menos 4 configurações à escolha do grupo
Variação de quadros	Opcional

Contagem: 2 cenários × 2 configurações × 7 variações (FCFS, SJF e RR com 5 quantums) = 28 execuções base.

Como apresentar os resultados:

- Uma tabela resumo por cenário, com uma linha por execução e uma coluna por métrica obrigatória.
- Pelo menos 3 gráficos, por exemplo: espera média por escalonador, efeito do quantum no RR (espera, resposta e trocas de contexto) e speedup por escalonador.
- Os dois Gantt (1 CPU e 4 CPUs) como figuras legíveis.
- Cada afirmação da análise deve citar a execução e a métrica que a sustenta.

As diretrizes de entrega e critérios avaliados serão definidas e estarão no AVA2.0